Software Engineering Makeup Exam

Name: _____

- Exam is open note, but closed computer - printed notes are fine, but no laptop/phone/smart watch

- Please write legibly!

- Partial answers get partial credit - explain your reasoning to get credit for all the parts of the answer you know

- Write valid Java code for questions asking for code; you **do not** need to include any import statements (assume the IDE will handle that for you)

1. Imagine you're given the following workflow that a SongPlayingService API must support:
    - Load a playlist of songs
    - Play specific songs from the playlist
    - Add a new song to the playlist

(a) Write a prototype for a client of SongPlayingService

(b) Using your answer from (a), fill out the SongPlayingService interface:

```
public interface SongPlayingService {
```

2. Imagine you're creating a library to help with a very flexible way to display some simple graphics. A program will specify its graphics in terms of Rectangles, Circles, and Lines (assume Rectangle, Circle, and Line are all defined interfaces), and then it can be displayed on any graphics frontend that can draw Rectangles, Circles, and Lines. You want your library to support many different types of graphics frontends - some might have an ASCII Art display, some might display graphics in a browser, some might use a Java applet, etc.

(a) What design pattern could you use to build your library?

(b) Write the code for (a) that would be part of your library and used by a graphics frontend - you do not need to write an implementation of a graphics frontend.

3. Imagine you need to serialize the following class. Write the proto message that would let you serialize the data in the class over the wire.

```
public class DataClass {
   private List<String> data;
   private int id;

   public List<String> getData() {
       return data;
   }

   public int getId() {
       return id;
   }
}
```

4. Consider the following code, which lets many people play a multithreaded game. Unfortunately, the code doesn't work - it often returns '0' even when the 0th player didn't win. More specifically, that's the **only** buggy symptom - the method sometimes returns the correct index of the winning player (which is sometimes 0), or sometimes incorrectly returns 0. Given that information, identify the bug and change the code to fix it.

```
public int playGame(int numPlayers) {

        AtomicInteger playerWon = new AtomicInteger(0);

        ExecutorService threadPool = Executors.newCachedThreadPool();
        for (int i = 0; i < numPlayers; i++) {
            // all variables used in a lambda must be effectively
            // final, i is not
            // this is not a bug
            int finalI = i;

            Callable<Void> playGame = () -> {
                while (!gameIsOver()) {
                    boolean hasWon = takeTurn();
                    if (hasWon) {
                        playerWon.set(finalI);
                    }
                }
                return null;
            };
            threadPool.submit(playGame);
        }

        return playerWon.get();
    }

    private boolean takeTurn() {
        // method definition omitted, there are no bugs here
    }

    private boolean gameIsOver() {
        // method definition omitted, there are no bugs here
    }
```

// Space for Q4

5. Normally, if you call put() on a Map with a key that already exists, the previous value gets silently overwritten. Suppose you want to be able to instead throw an exception if a key already exists, and you'd like to be able to do this with any Map in your code base.

(a) What design pattern could you use to do this?

(b) Write the code to apply the pattern from (a) to solve this problem

6. Consider the following two classes. Write a **unit test** for the Barn class - you may use any libraries we've covered in class.

```
public class Barn {

    private final List<Goat> goatsInBarn = new ArrayList<>();

    // Never have just one goat
    public Barn(Goat initialGoat1, Goat initialGoat2) {
        goatsInBarn.add(initialGoat1);
        goatsInBarn.add(initialGoat2);
    }

    public void feedGoats() {
        for (Goat g : goatsInBarn) {
            g.feedHay();
        }
    }
}

public interface Goat {
    public void feedHay();
}
```

7. Draw a picture of what the stack and heap would look like just after the last line of `exampleTable`, but before the method returns.

```
public class Table {

    private Chair[] chairs;

    public Table(int numChairs) {
        this.chairs = new Chair[numChairs];
        for (int i = 0; i < numChairs; i++) {
            chairs[i] = new Chair();
        }
    }

    public void personSitsAtTable(Person person) {
        for (Chair chair : chairs) {
            if (chair.isEmpty()) {
                person.sitInChair(chair);
            }
        }
    }

    public static void exampleTable() {
        Table table = new Table(2);
        table.personSitsAtTable(new Person());
        Person secondPerson = new Person();
        table.personSitsAtTable(secondPerson);
    }
}

public class Person {
    private Chair seat; // null if person is standing

    public void sitInChair(Chair seat) {
        this.seat = seat;
        seat.personSitsDown(this);
    }

    public void standUp() {
        seat.personStandsUp();
```

```
        seat = null;
    }
}

public class Chair {
    private Person inChair; // null if chair is empty

    public void personSitsDown(Person person) {
        this.inChair = person;
    }

    public void personStandsUp() {
        inChair = null;
    }
    public boolean isEmpty() {
        return inChair == null;
    }
}
```